

yt-agent2 — Code Review Report

Static analysis of all 7 project files

June 23, 2026

Each file was reviewed across six dimensions: syntax errors, logic errors, missing imports, unused imports, hardcoded values that should be environment variables, error handling gaps, and code that works locally but would break on Railway Linux. Issues are prioritised HIGH / MEDIUM / LOW.

File 1 of 7 — agents/transcript_agent.py

Syntax Errors

[OK] No syntax errors found.

Logic Errors

HIGH

`_get_cookies_path()` silently drops valid cookie content

Location: agents/transcript_agent.py, lines 195-206

The method only writes cookie content to a temp file if the value starts with '#' or '.'. A valid Netscape cookie file may start with '.youtube.com' (a domain line) or any other character. If the env var is set to raw cookie content that does not start with '#' or '.', the method returns None with no warning — cookies are silently ignored and bot detection is not bypassed. Since this is the current #1 unresolved blocker, this is the highest-priority issue.

Suggested fix: Widen the condition: check for the Netscape header explicitly, or treat any multi-line string (containing a tab character) as cookie content, regardless of its first character.

MEDIUM

`format_error` and `unknown` types retry without backoff or user feedback

Location: agents/transcript_agent.py, lines 98-121

When `_classify()` returns 'format_error' or 'unknown', the error falls through both the UNRECOVERABLE and BACKOFF_RETRY blocks. On non-final attempts the loop just continues immediately with no sleep. This means 3 rapid-fire retries with no pause, which wastes quota and could make transient errors worse.

Suggested fix: Add a small fixed sleep (e.g. 1s) before continuing on unknown/format errors.

Missing Imports

[OK] No missing imports. All used modules are imported.

Unused Imports

LOW

`import re` is imported but never used

Location: agents/transcript_agent.py, line 15

'import re' appears at the top of the file. No regular expressions are used anywhere in transcript_agent.py. This is dead code that adds unnecessary noise.

Suggested fix: Remove line 15: import re

Hardcoded Values

[OK] 'whisper-large-v3-turbo' model name is a class constant — acceptable as a deliberate model pin, not a configuration concern.

[OK] MAX_FILE_SIZE_MB=24 and MAX_RETRIES=3 are appropriate class constants.

Error Handling Gaps

LOW

Cookie temp file created with delete=False is never cleaned up

Location: `agents/transcript_agent.py`, lines 197-205

`_get_cookies_path()` writes cookie content to a `NamedTemporaryFile` with `delete=False`, then returns the path. The temp file is never deleted. In a long-running Railway container, this creates one orphaned temp file per request that uses cookies. The OS will clear `/tmp` on container restart, so this is low risk but technically a resource leak.

Suggested fix: Store the temp path and delete it after `yt-dlp` completes in `_download_audio()`.

Local vs Railway Linux

MEDIUM

`shutil.which('node')` returns a Windows path locally, None on Railway

Location: `agents/transcript_agent.py`, lines 127-128

On the developer's Windows machine, `shutil.which('node')` returns a valid path like `C:\Program Files\nodejs\node.exe`. The fallback `'/usr/bin/node'` is then checked with `os.path.exists()` — on Railway where Node is not installed, this returns `False` and `js_runtimes` becomes `{}`. This is handled correctly. However, the Windows path from `shutil.which()` would be passed to `os.path.exists()` first and return `True` on Windows, meaning `js_runtimes` is populated locally but empty on Railway. `yt-dlp` behaviour may differ between environments as a result.

Suggested fix: Explicitly check for the Linux node path or skip `js_runtimes` if not on a supported platform. Alternatively, add `node` to `nixpacks.toml` `nixPkgs`.

File 2 of 7 — agents/blog_post_agent.py

Syntax Errors

[OK] No syntax errors found.

Logic Errors

LOW

'401' in last_error string match is overly broad

Location: agents/blog_post_agent.py, line 150

The check 'if "401" in last_error' is meant to detect an HTTP 401 Unauthorized response. However, any error message that contains the substring '401' for any reason (e.g., 'Error code 4010', a token count that happens to include 401, or a model version string) would incorrectly trigger this branch and raise 'Invalid Groq API key.' immediately. While unlikely to cause a false positive in practice, it is fragile.

Suggested fix: Check for 'status_code == 401' or 'AuthenticationError' exception type from the Groq SDK instead of string matching.

Missing Imports

[OK] No missing imports.

Unused Imports

[OK] No unused imports. os, time, Groq, and load_dotenv are all used.

Hardcoded Values

[OK] MODEL = 'llama-3.3-70b-versatile' is a class constant — correct. Changing it is a deliberate action, not a configuration concern.

[OK] max_tokens values (1500, 2000) and temperatures (0.4, 0.7) are appropriately hardcoded as tuned parameters.

Error Handling Gaps

MEDIUM

No timeout on Groq API calls — requests can hang indefinitely

Location: agents/blog_post_agent.py, lines 132-140

The client.chat.completions.create() call has no timeout parameter. If Groq's API becomes slow or unresponsive, the call will hang indefinitely, blocking the Streamlit session with no progress feedback and no way for the user to cancel. The same issue exists in TranscriptAgent._transcribe_with_retry().

Suggested fix: Pass a timeout value (e.g. timeout=120) to the Groq client call, or configure it on the Groq() client constructor with httpx timeout settings.

LOW

No guard against Groq context-length errors for very long transcripts

Location: agents/blog_post_agent.py, line 83 and 118

_truncate() limits the transcript sent to the LLM, but very long research notes from the first LLM call are passed in full to the second call's user_prompt with no truncation. If research_notes exceeds several thousand tokens, combined with system_prompt and the transcript excerpt, the total prompt could approach or exceed Groq's context window, causing a context length error that is not specifically handled.

Suggested fix: Add a _truncate(research_notes, 3000) before interpolating into the write prompt.

Local vs Railway Linux

[OK] No environment-specific behaviour. All library calls (Groq SDK, standard Python) are cross-platform.

File 3 of 7 — orchestrator.py

Syntax Errors

[OK] No syntax errors found.

Logic Errors

[OK] No logic errors. Sequential agent calls are correct and safe.

Missing Imports

[OK] No missing imports.

Unused Imports

[OK] No unused imports.

Hardcoded Values

[OK] No hardcoded values requiring externalisation.

Error Handling

[OK] No errors are caught by design — all propagate to app.py. Correct.

Local vs Railway

[OK] No environment-specific code. Fully portable.

File 4 of 7 — app.py

Syntax Errors

[OK] No syntax errors found.

Logic Errors

HIGH

blog_post_to_docx() runs on every Streamlit re-render, not just on download

Location: app.py, line 106

The call `blog_post_to_docx(blog_post)` is passed directly as the 'data' argument to `st.download_button()`. Streamlit re-executes the entire script on every user interaction (tab switch, scroll, any widget change). This means a full Word document is built in memory on every re-render after a successful generation — not only when the download button is actually clicked. For typical blog posts this is fast but wasteful; under load or with large documents it degrades UX noticeably.

Suggested fix: Compute the docx bytes once after the pipeline completes and store in a variable: `docx_bytes = blog_post_to_docx(blog_post)`. Then pass `data=docx_bytes`.

Missing Imports

[OK] No missing imports. All used modules are imported.

Unused Imports

LOW

from docx.shared import Pt is imported but Pt is never used

Location: app.py, line 5

Pt is imported from `docx.shared` but is not referenced anywhere in `app.py`. The `blog_post_to_docx()` function does not set any font sizes.

Suggested fix: Remove 'from docx.shared import Pt' from line 5.

Hardcoded Values

[OK] No hardcoded values requiring externalisation.

Error Handling Gaps

MEDIUM

blog_post_to_docx() has no error handling — exceptions surface as raw UI errors

Location: app.py, lines 9-24

If `python-docx` raises any exception during document creation (malformed content, memory error, library bug), it propagates uncaught through the `st.download_button data=` argument and displays a raw Python traceback to the user rather than a friendly error message.

Suggested fix: Wrap the body of `blog_post_to_docx()` in a `try/except`, or wrap the call site in a `try/except` that returns a placeholder bytes object and shows `st.warning()`.

MEDIUM

GROQ_API_KEY warning shown but pipeline still runs — produces confusing error

Location: `app.py`, lines 44-46 and 49

The `GROQ_API_KEY` check shows a `st.warning()` at page load, but does not prevent the Generate button from running the pipeline. If a user dismisses the warning and clicks Generate anyway, the error surfaces deep inside `TranscriptAgent.__init__()` as `'GROQ_API_KEY not found. Add it to your .env file.'` — a developer-facing message that is confusing in a deployed context.

Suggested fix: In the 'if generate:' block, re-check `os.environ.get('GROQ_API_KEY')` and call `st.error()` + `st.stop()` before instantiating the Orchestrator.

Local vs Railway Linux

[OK] Monkey-patching of agent `.run()` methods is safe on both platforms — each Streamlit user session is an independent Python execution context.

File 5 of 7 — requirements.txt

Contents Review

```
yt-dlp==2026.6.9
groq==1.4.0
python-dotenv==1.2.2
streamlit==1.41.1
python-docx==1.1.2
```

HIGH

yt-dlp==2026.6.9 — verify this exact version exists on PyPI

Location: requirements.txt, line 1

yt-dlp releases very frequently. Version 2026.6.9 follows the YYYY.M.D naming scheme. If this exact version does not exist on PyPI (e.g., the release was 2026.6.8 or 2026.6.10), Railway's pip install will fail with 'No matching distribution found' and the build will not complete. This would take the entire deployment down silently.

Suggested fix: Run 'pip index versions yt-dlp' locally or check <https://pypi.org/project/yt-dlp/> to confirm 2026.6.9 exists. If not, update to the nearest valid version.

[OK] groq==1.4.0 — valid, confirmed release.

[OK] python-dotenv==1.2.2 — valid, confirmed release.

[OK] streamlit==1.41.1 — valid, confirmed release.

[OK] python-docx==1.1.2 — valid, confirmed release.

[OK] No Windows-only packages present. Safe to install on Railway's Linux container.

File 6 of 7 — railway.toml

Contents Review

```
[build]
builder = "nixpacks"
```

```
[deploy]
startCommand = "PYTHONPATH=/app streamlit run app.py --server.port $PORT --server.address 0.0.0.0 --server.headless true"
healthcheckPath = "/_stcore/health"
healthcheckTimeout = 300
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 3
```

MEDIUM

restartPolicyType = 'ON_FAILURE' — casing should be verified against Railway schema

Location: railway.toml, line 8

Railway previously rejected the lowercase variant 'on-failure' with a parse error. The current value 'ON_FAILURE' was set based on that experience. Railway's schema documentation shows the accepted values as 'ALWAYS', 'ON_FAILURE', and 'NEVER' in uppercase — so the current value appears correct. However, this should be confirmed by checking Railway's latest TOML spec, as their schema has changed before.

Suggested fix: Check Railway docs at docs.railway.app for current railway.toml schema. If in doubt, remove the line — Railway applies a sensible default.

[OK] PYTHONPATH=/app — correctly set. Required for agents/ module imports.

[OK] --server.headless true — correctly set for server environments.

[OK] healthcheckPath = '/_stcore/health' — correct Streamlit health endpoint.

[OK] healthcheckTimeout = 300 — 5 minutes is appropriate for cold start.

[OK] No buildCommand — correct, Nixpacks auto-detects requirements.txt.

File 7 of 7 — nixpacks.toml

Contents Review

```
[phases.setup]
nixPkgs = ["ffmpeg", "python312"]
```

[OK] No syntax errors. Valid TOML.

[OK] ffmpeg — correctly included. Required by yt-dlp FFmpegExtractAudio post-processor.

[OK] python312 — correctly included. Pins Python version to avoid environment mismatches.

[OK] No missing packages for current requirements.txt dependencies.

[OK] No unused packages.

LOW

node is not in nixPkgs — js_runtimes is always empty on Railway

Location: nixpacks.toml, line 2

transcript_agent.py builds a `js_runtimes` dict using `shutil.which('node')`, falling back to `'/usr/bin/node'`. On Railway, neither exists, so `js_runtimes` is always `{}`. The `js_runtimes` option is passed to yt-dlp but has no effect. If yt-dlp ever requires a JS runtime for a fallback path, it will fail silently. Adding node to `nixPkgs` would make the runtime available and `js_runtimes` functional on Railway.

Suggested fix: Add 'nodejs' to nixPkgs: `nixPkgs = ["ffmpeg", "python312", "nodejs"]`

Prioritised Issue List — All Files

The following table lists every issue found across all 7 files, ranked by severity. HIGH issues will break functionality or a critical feature. MEDIUM issues degrade UX, robustness, or correctness under specific conditions. LOW issues are code quality improvements.

Priori ty	File	Issue	Impact / Risk
HIGH	requirements.txt	yt-dlp==2026.6.9 may not exist on PyPI	Entire Railway build fails -- app does not deploy
HIGH	transcript_agent.py	_get_cookies_path() silently drops valid cookie content	Bot detection bypass (current #1 blocker fix) does not work
HIGH	app.py	blog_post_to_docx() runs on every Streamlit re-render	Word doc rebuilt in memory on every user interaction after generation
MED	app.py	GROQ_API_KEY missing: warning shown but pipeline still runs	User gets confusing internal error instead of a clear message
MED	app.py	blog_post_to_docx() has no error handling	Any docx failure shows raw Python traceback in the UI
MED	blog_post_agent.py	No timeout on Groq API calls	Slow/unresponsive API hangs the session indefinitely
MED	transcript_agent.py	format_error/unknown retries with no backoff or pause	3 rapid-fire retries waste quota, may worsen transient errors
MED	transcript_agent.py	js_runtimes always empty on Railway	yt-dlp JS fallback path unavailable; local vs Railway diverge
MED	railway.toml	restartPolicyType casing -- verify against current schema	Wrong casing causes Railway to reject the config file
LOW	transcript_agent.py	import re is unused	Dead code -- minor code quality issue
LOW	transcript_agent.py	Cookie temp file never cleaned up	Orphaned /tmp files accumulate during long-running sessions
LOW	blog_post_agent.py	'401' in last_error string match is fragile	Could false-positive on messages containing '401'
LOW	blog_post_agent.py	research_notes not truncated before write prompt	Very long notes could push prompt past context limit
LOW	app.py	from docx.shared import Pt is unused	Dead import -- minor code quality issue
LOW	nixpacks.toml	nodejs not in nixPkgs -- js_runtimes empty on Railway	yt-dlp JS runtime unavailable; js_runtimes config has no effect

Summary: 3 HIGH | 6 MEDIUM | 6 LOW | 16 total issues

End of Report